

Artifact Description

The artifact selected for this is once again the same Android inventory app used in the first and second category from CS 360: Mobile Architecture and Programming. The original application was a simple inventory management app that stored items in an SQLite table and displayed them to the user in a simple list using a GridLayout. It had a class called DatabaseManager that had two tables: one for users with username and plain-text password columns, and another for an inventory table with names, quantities, and a user ID foreign key. The database had no category structure, no secure credential storage, and it was designed around simple, generic inventory use. This artifact was selected for the Database category because it presented a clear opportunity to demonstrate relational database design, schema normalization, and security-minded data handling through documented evolution of the preexisting code.

Justification for Inclusion

The artifact was selected because the original database design had several significant weaknesses that could be directly addressed through professional database enhancements. The first and most critical issue was that the user passwords were stored and compared as plain text. If the SQLite database file were to be accessed by an unauthorized party, every user's password would be immediately readable. This is a fundamental security vulnerability that no production application should have.

The second issue was that the schema was not designed for a media library domain. The original inventory table had only item_name and quantity columns, neither of which were relevant to cataloging books, movies, or games. A proper media library requires fields for titles, categories, years, ratings, and notes. The categories themselves should be enforced using a foreign key relationship that separates category tables rather than storing them as simple text strings. Using plain text would allow for typos and inconsistencies that could create new, invisible categories, destroying the tab-based navigation set up in the UI.

To address these issues, several enhancements we made. The schema was redesigned into three tables: users, categories, and media_items. The categories table is seeded with Books, Movies, and Games on database creation, and the media_items table references it through a foreign key on category_id, enforcing integrity at the database level. The users table was updated to include a password_salt column alongside the password column. Password hashing was implemented using SHA-256 with a randomly generated salt per each user. When a user creates an account, a 16-byte cryptographically random salt is generated using SecureRandom, the password is linked with the salt and hashed using SHA-256, and both the hash and the salt are

stored in the database. The plain-text password is never stored anywhere. When a user logs in, the stored salt is retrieved, the input password is hashed with it, and the resulting hash is compared to the stored hash. This method ensures that two users with the same password will have completely different hashes in the database, preventing rainbow table attacks where a predetermined table of common password hashes is used to reverse engineer credentials. All database queries through the application use parameterized statements, preventing SQL injection attacks by ensuring that the user input is always treated as data rather than executable SQL.

Course Outcomes

The planned enhancement for Category Three targeted course outcome four: demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals (software engineering/design/database). This outcome is met through the redesign of the database to properly model the media library domain, the implementation of a normalized three-table relational structure with enforced foreign key constraints, and the addition of category filtering and search capabilities that deliver direct value to the application's functionality.

The enhancement also meets course outcome number five: develop a security mindset that anticipates adversarial exploits in software architecture and designs to expose potential vulnerabilities, mitigate design flaws, and ensure privacy and enhanced security of data and resources. The plain text password storage was a direct source of security vulnerability. Replacing it with SHA-256 hashing and using salts per user demonstrates an understanding of how credential data can be exposed and how to mitigate that risk. The use of parameterized queries throughout the database layer demonstrates an understanding of SQL injection as an attack vector and the established defense against it.

Process Reflection

One of the most significant learning points from the Category Three enhancement came from implementing password hashing correctly, rather than just conceptually. It was easy to understand that passwords should be hashed and not stored as plain text. However, actually implementing it required making several key decisions: which hashing algorithm to use, how to generate the salt, where to store the salt, and how to structure the login check so that the comparison is done correctly. SHA-256 was chosen because it is cryptographically secure, widely supported and documented, and guaranteed to be available on all Android devices through the Java MessageDigest API. The salt was generated using SecureRandom rather than the standard Random class because SecureRandom is designed for cryptographic use and produces values that are not predictable from previous outputs, which is necessary for a salt.

One challenge I encountered during this enhancement was that adding the password_salt column to the users table required incrementing the database version number and relying on the

onUpgrade method to drop and recreate all tables. This meant that any existing test accounts created without hashing were deleted when the app was reinstalled or the database was upgraded. In a production application, this would require a migration strategy that preserves existing data while adding the new column, but for the purpose of this project the recreation approach was acceptable since no real user data was at risk. It was also necessary to update the API level from 24 to 26 to use more updated security measures. This includes using the getEncoder from Base64.

Across all three enhancement categories, this capstone project transformed a functional but limited inventory app into a structured, secure, and professionally designed media library. The database layer specifically evolved from a two-table schema with plain text credentials and a generic domain model into a normalized three table structure with hashed credentials, enforced data integrity, and query methods tailored to the application's needs. Each enhancement was grounded in established database and security principles, and the challenges I faced along the way helped produce a deeper understanding of why those principles exist.